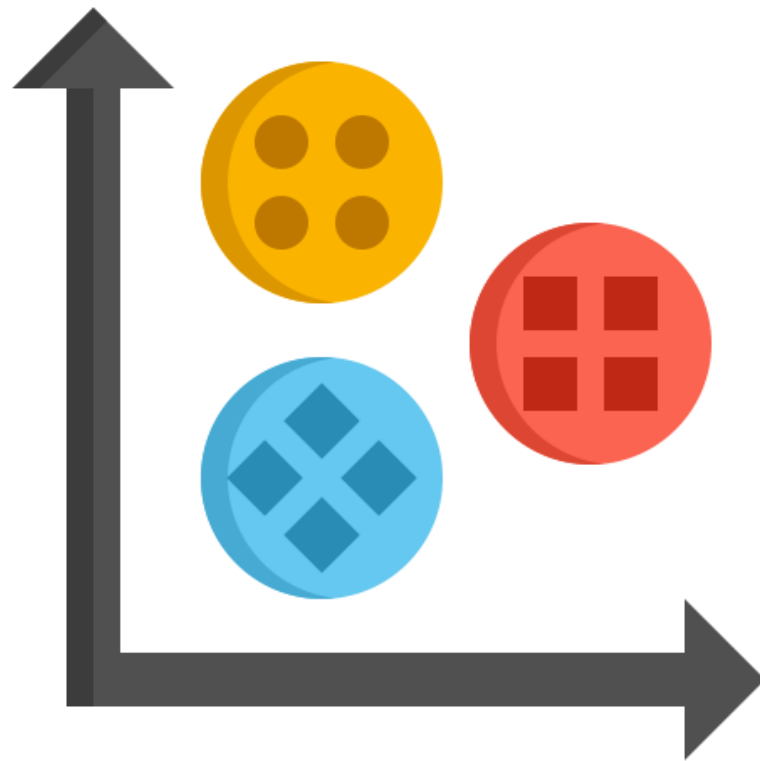


Clustering



Clustering

- **Clustering:** A type of **Unsupervised Learning** where the goal is to group similar data points together without pre-existing labels
- **Objective:** Maximize **intra-cluster** similarity (compactness) and minimize **inter-cluster** similarity (separation)

Key Concepts

- **Distance Metrics:** The foundation of similarity (e.g., **Euclidean** for geometry, **Manhattan** for grid, **Cosine** for text)
- **Cluster Structure:** Are clusters spherical (compact blobs) or arbitrary shapes (snakes, crescents)?
- **Density:** The number of points in a given unit volume

Applications

- **Segmenting customers** into groups with similar demographics or buying patterns for targeted marketing campaigns
- **Detecting anomalous behavior**, such as unauthorized network intrusions, by identifying patterns of use falling outside the known clusters
- **Simplifying large datasets** by grouping features with similar values into a smaller number of homogeneous categories

K-Means Algorithm

How it Works (Iterative Optimization)

1. **Initialization:** Select K random points as initial centroids (typically using **k-means++** to spread them out).
2. **Assignment Step:** Assign each data point to its closest centroid based on Euclidean distance.
3. **Update Step:** Recalculate centroids by taking the mean of all points assigned to that cluster.
4. **Convergence:** Repeat until centroids do not change or change is below a tolerance threshold.

Mathematical Objective

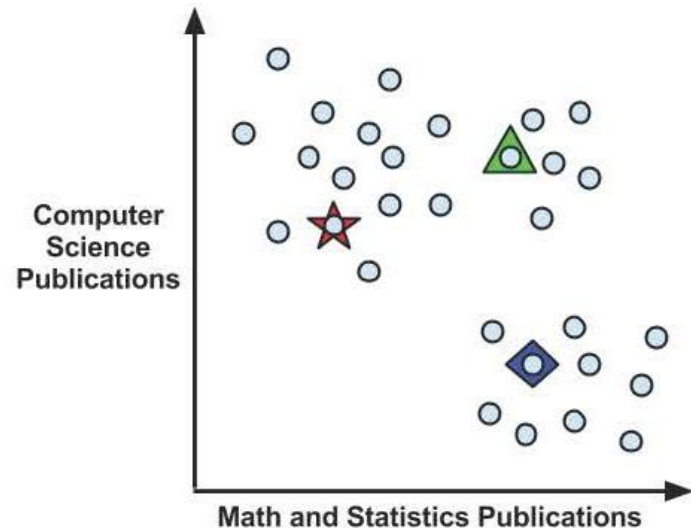
- Minimizes **Inertia** (Within-Cluster Sum of Squares - WCSS):
- $$\sum_{i=0}^n \min_{\mu_j \in C} (||x_i - \mu_j||^2)$$
- **Intuition:** It tries to make clusters as compact (spherical) as possible.

Usage Scenarios

- **When to use:** You know K , clusters are roughly spherical, equal variance, and similar size.
- **Examples:** Market Segmentation, Document categorization.

Using distance to assign and update clusters

- k-means treats feature values as coordinates in a multidimensional feature space.
- It starts by selecting k initial centroids—typically chosen randomly from the dataset
- Each point is then assigned to its nearest centroid, after which the centroids are updated as the mean of their assigned points.
- This iterative process continues until the cluster assignments stabilize.

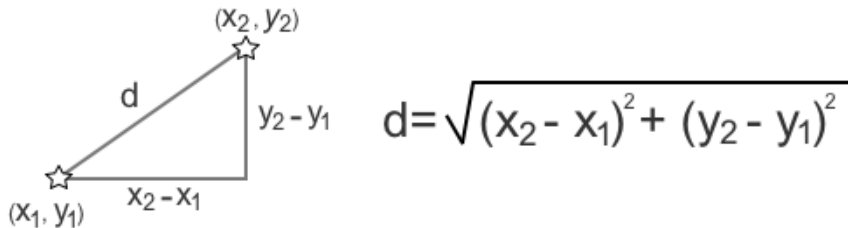


Euclidean distance

- Euclidean distance measures how far two points are in a geometric space
- For two points: $x = (x_1, x_2, \dots, x_n)$, $y = (y_1, y_2, \dots, y_n)$

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2}$$

- k-means relies on Euclidean distance to decide **cluster membership** and **update centroids**



K-Means: Pros & Cons

✓ Benefits

- **Efficiency:** Very fast, generally ($O(n)$) (Linear scaling with n)
- **Easy Implementation:** Standard baseline for clustering
- **Scalable:** Works well on large datasets
- **Interpretable:** Centroids represent the “average” of the cluster

⚠ Limitations

- **Need to choose K:** The number of clusters must be specified upfront
- **Sensitive to Initialization:** Bad starting points can lead to poor results (mitigated by `k-means++`)
- **Spherical Assumption:** Fails on irregular shapes (e.g., crescents) or clusters with different densities/sizes
- **Sensitive to Outliers:** Outliers can pull centroids away from the true center
- **Curse of Dimensionality:** In high dimensions, Euclidean distance becomes less discriminative (distances converge)

K-Means Implementation



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate synthetic data with distinct blobs
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Implementation
# init='k-means++': Smarter initialization to speed up convergence
# n_init=10: Run algorithm 10 times with different seeds, pick best result (lowest inertia)
kmeans = KMeans(n_clusters=4, init='k-means++', n_init=10, random_state=42)
y_pred = kmeans.fit_predict(X)

# Visualization
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_pred, s=50, cmap='viridis', edgecolors='k')
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.9, marker='x',
label='Centroids')
plt.title("K-Means Partitioning (Voronoi Cells)")
plt.legend()
plt.show()
```


Choosing K: The Elbow Method

Determining the Optimal Number of Clusters

- **Concept:** Plot **Inertia** (sum of squared distances) vs. number of clusters K
- **Inertia Behavior:** Decreases monotonically as K increases. At $K = N$, inertia is 0 (every point is a cluster).
- **The “Elbow”:** The point of diminishing returns where adding a cluster doesn’t significantly reduce variance.

```
inertia = []
K_range = range(1, 10)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

plt.plot(K_range, inertia, 'bx-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.title('Elbow Method: Look for the kink')
plt.show()
```

Choosing K: Silhouette Analysis

Validating Cluster Consistency

- **Concept:** Measures how similar a point is to its own cluster (**cohesion**, a) compared to other clusters (**separation**, b).

- $$S = \frac{b-a}{\max(a,b)}$$

- **Interpretation:**

- **+1:** Perfect clustering (dense and well-separated).
- **> 0.7:** Strong structure.
- **0.5 - 0.7:** Reasonable structure.
- **< 0.25:** No substantial structure.
- **0:** Point is on decision boundary.
- **Negative:** Assigned to wrong cluster.

```
from sklearn.metrics import silhouette_score  
  
# A higher mean score indicates better defined clusters  
score = silhouette_score(X, kmeans.labels_)  
print(f"Silhouette Score: {score:.3f}")
```

- Silhouette is computationally expensive ($O(N^2)$) compared to Inertia.
- Useful for comparing different K values (pick K maximizing Silhouette).

DBSCAN Algorithm

Density-Based Spatial Clustering of Applications with Noise

How it Works (Density Reachability)

DBSCAN defines clusters as continuous regions of high density separated by regions of low density. It requires two parameters:

- **eps (ϵ):** The maximum distance for neighborhood radius
- **min_samples:** The minimum number of points required to form a dense region.

Point Classifications

- **Core Point:** Has $\geq \text{min_samples}$ within eps radius. (Interior of cluster).
- **Border Point:** Within eps of a Core Point but has $< \text{min_samples}$. (Edge of cluster).
- **Noise Point:** Neither Core nor Border. (Outlier).

Intuition

- **“Friends of Friends”:** If A is close to B, and B is close to C, they chain together into one cluster if density is sufficient.
- **Robust:** Can discover any shape (not just circles).
- Crucial distinction: K-Means partitions *all* space. DBSCAN leaves “empty” space as noise.

DBSCAN: Pros & Cons

✓ Benefits

- **Shape Agnostic:** Can find clusters that are nested, crescent-shaped, or linear.
- **Noise Handling:** Unlike K-Means, outliers do not distort the cluster (they are labeled -1).
- **No K Required:** Number of clusters is determined by the data density.
- **Few Parameters:** Generally only `eps` and `min_samples` to tune.

⚠ Limitations

- **Varying Densities:** Fails if clusters have significantly different densities (cannot find a single `eps` that works for all).
- **Parameter Sensitivity:** Finding the right `eps` can be tricky.
 - Too small `eps` → Everything is noise.
 - Too large `eps` → Everything merges into one blob.
- **High Dimensions:** “Curse of Dimensionality” implies Euclidean distance is less meaningful, making “radius” hard to define.

DBSCAN Implementation & Tuning



Heuristic for `min_samples`: Often set to $D \times 2$ (where D is dimensions). For 2D data, `min_samples=4` is a good start.

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

# Generate non-spherical data (interleaved half-circles)
X, _ = make_moons(n_samples=300, noise=0.05, random_state=42)

# Implementation
# eps typically requires domain knowledge or k-distance graph
dbscan = DBSCAN(eps=0.3, min_samples=5)
y_pred = dbscan.fit_predict(X)

# Visualization
plt.figure(figsize=(8, 6))
unique_labels = set(y_pred)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]

for k, col in zip(unique_labels, colors):
    if k == -1:
        # Black used for noise.
        col = [0, 0, 0, 1]
        label = "Noise (Outliers)"
    else:
        label = f"Cluster {k} (Core+Border)"

    class_member_mask = (y_pred == k)
    xy = X[class_member_mask]

    # Plotting
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col),
             markeredgecolor='k', markersize=10 if k != -1 else 6, label=label)

plt.title("DBSCAN: Handling Arbitrary Shapes & Noise")
plt.legend()
plt.show()
```

Model Evaluation Metrics

Since we lack Ground Truth labels, we use **Internal Validation**.

1. Silhouette Coefficient (Range: -1 to 1)

- Measures: **Cohesion** (closeness to own cluster) vs **Separation** (distance to next closest)
- Best for: Convex/Spherical clusters (K-Means)
- If Silhouette is high, clusters are dense and far apart
- **Caveat:** Can be misleading for DBSCAN. A perfect crescent shape might have a lower silhouette score than a mediocre circle because points at the ends of the crescent are “far” from the center in Euclidean terms.

2. Davies-Bouldin Index (DBI) (Lower is better)

- Measures: Average similarity ratio of each cluster with its most similar cluster
- **Calculation:** Ratio of “within-cluster scatter” to “between-cluster separation”.
- **Lower score** = Better separation
- **Logic:** Clusters should be tight (low scatter) and far apart (high separation).

3. Calinski-Harabasz Score (Variance Ratio Criterion) (Higher is better)

- **Calculation:** Ratio of between-cluster dispersion to within-cluster dispersion.
- **Goal:** Maximize score. Higher is better.
- **Pros:** Fast to compute.

Metric Interpretation Code



```
from sklearn.metrics import silhouette_score, davies_bouldin_score,
calinski_harabasz_score

def evaluate_clustering(X, labels):
    if len(set(labels)) > 1:
        # Silhouette: -1 to 1 (Higher is better)
        s = silhouette_score(X, labels)

        # Davies-Bouldin: 0 to inf (Lower is better)
        db = davies_bouldin_score(X, labels)

        # Calinski-Harabasz: 0 to inf (Higher is better)
        ch = calinski_harabasz_score(X, labels)

        print(f"Silhouette: {s:.3f} (Structure strength)")
        print(f"Davies-Bouldin: {db:.3f} (Separation quality - Lower better)")
        print(f"Calinski-Harabasz: {ch:.3f} (Variance ratio - Higher better)")
    else:
        print("Error: Not enough clusters to evaluate.")

evaluate_clustering(X, y_pred)
```

Guidelines

1. **Exploratory Phase:** Run **K-Means** first for a baseline
2. **Complex Data:** If K-Means fails or data has noise, switch to **DBSCAN**
3. **Validation:** Always inspect visually (if possible) and use Silhouette score with caution regarding shape assumptions.